

```
"""
```

```
SemanticFriction - dH/dt accumulator tracking the actual GENERIC Hamiltonian.
```

```
WHAT CHANGED:
```

```
Previous versions tracked dE/dt where  $E = |z|^2/2$  (DC mode proxy).
```

```
This version tracks dH/dt where  $H = H[\psi]$  (actual GENERIC Hamiltonian).
```

```
The friction and the relativistic outputs (MSR stress tensor, Einstein tensor) are now tracking the SAME physical quantity. The bridge between semantic closure and GR is now operative, not just structural.
```

```
 $H[\psi] = \int [ \alpha |\nabla\psi|^2 - \mu |\psi|^2 + (g/2) |\psi|^4 ]$  (full GENERIC Hamiltonian)
```

```
dH/dt = (H_curr - H_prev) / dt (energy flow rate)
```

```
Closure when  $|dH/dt| < dH\_dt\_floor$ :
```

```
The system's energy rate has dropped below the mean thermal fluctuation rate of  $H[\psi]$  - indistinguishable from equilibrium.
```

```
This is the same condition the MSR stress tensor measures.
```

```
THRESHOLDS (from 50 GENERIC trajectories, 2000 production steps each):
```

```
H_target = 1.0928 mean( $H[\psi]$ )
```

```
dH_dt_floor = 1.2923 mean( $|dH/dt|$ ) - thermal noise rate of H
```

```
dH_dt_p25 = 0.5192 lower quartile (tighter threshold)
```

```
PATTERN (Engine3 structure preserved):
```

```
H = client.dynamics.hamiltonian(lift(z)) actual energy
```

```
dH_dt = (H - H_prev) / dt rate of change
```

```
friction -= step_weight / (1 +  $|dH\_dt|$  / floor) decimal walk
```

```
closed when friction < 0.15 AND  $|dH\_dt| < floor$  AND  $|z-z^*| < tol$ 
```

```
"""
```

```
from __future__ import annotations
```

```
import numpy as np
```

```
from typing import List, Optional, Callable
```

```
from dataclasses import dataclass
```

```
from uma.core.state import FieldPosterior
```

```
from uma.semantic.constants import (
```

```
    CALIBRATED_H_TARGET,
```

```
    CALIBRATED_H_STD,
```

```
    CALIBRATED_DH_DT_FLOOR,
```

```
    CALIBRATED_DH_DT_P25,
```

```
    CALIBRATED_FIELD_SCALE,
```

```
)
```

```

# -----
# Config
# -----

@dataclass
class SemanticFrictionConfig:
    step_weight: float = 0.02
    closure_friction_threshold: float = 0.15
    convergence_dz_dt: float = CALIBRATED_DH_DT_FLOOR
    closure_state_tol: float = CALIBRATED_FIELD_SCALE * 2.0
    min_steps_before_closure: int = 8
    kT: float = 0.005
    lam: float = 0.4
    N: int = 32
    dt: float = 0.04

    @classmethod
    def from_uma_config(cls, cfg, dt: float = 0.04, **kwargs) -> "SemanticFriction":
        return cls(kT=cfg.generic.kT, lam=cfg.generic.lam,
                   N=cfg.grid.N, dt=dt, **kwargs)

# -----
# FrictionRecord
# -----

@dataclass
class FrictionRecord:
    t: float
    H: float          # actual  $H[\psi]$  – real Hamiltonian
    E: float          #  $|z|^2/2$  – proxy (kept for reference)
    dH_dt: float      #  $(H - H_{\text{prev}}) / dt$  – energy flow rate
    dE_dt: float      #  $(E - E_{\text{prev}}) / dt$  – proxy rate (reference)
    friction: float   # accumulated  $\epsilon \in [0,1]$ 
    z_distance: float
    closed: bool

# -----
# SemanticFriction
# -----

class SemanticFriction:
    """
    Tracks semantic friction via  $dH/dt$  – the actual GENERIC Hamiltonian rate.

```

Requires a `hamiltonian_fn: z -> H[ψ]` callable.

This is provided by the pipeline from `client.dynamics.hamiltonian(lift(z))`.

When `hamiltonian_fn` is `None`, falls back to $E = |z|^2/2$ (proxy mode).

"""

```
def __init__(
    self,
    z_target: np.ndarray,
    config: Optional[SemanticFrictionConfig] = None,
    hamiltonian_fn: Optional[Callable[[np.ndarray], float]] = None,
):
    self.z_target = np.asarray(z_target, dtype=float)
    self.config = config or SemanticFrictionConfig()
    self.hamiltonian_fn = hamiltonian_fn # z -> H[ψ], or None for proxy mode

    self._friction: float = 1.0
    self._H_prev: Optional[float] = None
    self._E_prev: Optional[float] = None
    self._records: List[FrictionRecord] = []
    self._times: List[float] = []
    self._frictions: List[float] = []

def reset(self, initial_posterior: FieldPosterior) -> None:
    self._friction = 1.0
    self._H_prev = None
    self._E_prev = None
    self._records.clear()
    self._times.clear()
    self._frictions.clear()

def update(self, posterior: FieldPosterior, t: float) -> FrictionRecord:
    """
    One friction step.

    Tracks dH/dt if hamiltonian_fn is set, else dE/dt (proxy fallback).
    The convergence floor is always against the H-calibrated threshold.
    """
    cfg = self.config
    z = posterior.mean

    # Proxy energy (always computed for reference)
    E = float(np.real(z @ z) / 2.0)
    dE_dt = (E - self._E_prev) / cfg.dt if self._E_prev is not None else 0.0
    self._E_prev = E
```

```

# Actual Hamiltonian (if available)
if self.hamiltonian_fn is not None:
    H = self.hamiltonian_fn(z)
    dH_dt = (H - self._H_prev) / cfg.dt if self._H_prev is not None else
    self._H_prev = H
else:
    # Proxy fallback: scale E to H units
    # H_target / E_target ratio  $\approx$  H_mean / E_mean
    H = E * (CALIBRATED_H_TARGET / max(abs(E), 1e-15))
    dH_dt = dE_dt * (CALIBRATED_H_TARGET / max(abs(E), 1e-15))

# Friction decrement: driven by dH/dt against H-calibrated floor
floor = cfg.convergence_dz_dt + 1e-15
step = cfg.step_weight / (1.0 + abs(dH_dt) / floor)
self._friction = max(0.0, self._friction - step)

field_scale = CALIBRATED_FIELD_SCALE + 1e-15
z_dist = float(np.linalg.norm(z - self.z_target)) / field_scale

closed = False
if len(self._records) >= cfg.min_steps_before_closure:
    closed = (
        self._friction < cfg.closure_friction_threshold and
        abs(dH_dt) < cfg.convergence_dz_dt and
        z_dist < cfg.closure_state_tol / field_scale
    )

rec = FrictionRecord(
    t=t, H=H, E=E,
    dH_dt=dH_dt, dE_dt=dE_dt,
    friction=self._friction,
    z_distance=z_dist,
    closed=closed,
)
self._records.append(rec)
self._times.append(t)
self._frictions.append(self._friction)
return rec

@property
def value(self) -> float:
    return self._friction

@property
def is_closed(self) -> bool:
    return bool(self._records and self._records[-1].closed)

```

```

@property
def closure_step(self) -> Optional[int]:
    for i, r in enumerate(self._records):
        if r.closed:
            return i
    return None

@property
def records(self) -> List[FrictionRecord]:
    return list(self._records)

def summary(self) -> dict:
    if not self._records:
        return {"status": "no_data"}
    final = self._records[-1]
    return {
        "sf_total": round(self._friction, 6),
        "friction_final": round(final.friction, 6),
        "H_final": round(final.H, 6),
        "H_target": round(CALIBRATED_H_TARGET, 6),
        "dH_dt_final": round(final.dH_dt, 6),
        "dH_dt_floor": round(self.config.convergence_dz_dt, 6),
        "E_final": round(final.E, 8),
        "dE_dt_final": round(final.dE_dt, 8),
        "z_distance_final": round(final.z_distance, 4),
        "n_steps": len(self._records),
        "is_closed": self.is_closed,
        "closure_step": self.closure_step,
        "semantic_loss_pct": round(self._friction * 100, 2),
        "using_hamiltonian": self.hamiltonian_fn is not None,
    }

```